

Esercizio 1) (15 punti)

Si consideri il seguente server TCP per la prenotazione di un tavolo alla tavola calda del Dipartimento di Informatica (sede di Crema) implementato tramite le socket. Il server implementa due funzioni una per la prenotazione di un tavolo e una per la cancellazione di una prenotazione esistente. La funzione di prenotazione (**prenota**) prende in ingresso una stringa contenente il nome della prenotazione (*nome*), una stringa con il numero di persone (*numero*) e una stringa con l'orario e la data della prenotazione (*dataora*), e ritorna in uscita un codice a 10 caratteri (*codice*). La funzione di cancellazione (**cancella**) riceve in ingresso un codice a 10 caratteri (*codice_canc*) e ritorna in uscita una stringa che descrive lo stato della prenotazione (*stato*).

```
//DEFINIZIONE VARIABILI
```

```
[...]
```

```
//INIZIALIZZAZIONE STRUTTURE DATI
```

```
[...]
```

```
server = socket(AF_INET, SOCK_STREAM, 0);
```

```
bind(server, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr));
```

```
while(true)
```

```
{
```

```
    len = sizeof(caddr);
```

```
    client = connect(server, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr));
```

```
    pid= fork();
```

```
    if (pid!=0) {
```

```
        close(client);
```

```
    }
```

```
    else {
```

```
        close(server);
```

```
        op = recvfrom(client, operazione, MAX, 0, (struct sockaddr *) &echoClntAddr, &cliLen);
```

```
        if (op == "prenota") {
```

```
            recv(client, nome, len_nome,0);
```

```
            recv(client, numero, len_numero,0);
```

```
            recv(client, dataora, len_dataora,0);
```

```
            codice = prenota(nome,numero,dataora);
```

```
            send(codice);
```

```
        }
```

```
        elseif (op == "cancella") {
```

```
            recv(client, codice_canc, len_codice_canc,0);
```

```
            stato = cancella(codice_canc);
```

```
            send(stato);
```

```
        }
```

```
        close(client);
```

```
        exit(0);
```

```
    }
```

```
}
```

Si richiede di:

1. spiegare il codice presentato;
2. correggere eventuali errori nel codice presentato, giustificando ogni correzione;
3. fornire un'implementazione in pseudocodice di un server con socket TCP iterativo equivalente al server presentato;
4. discutere cosa cambia nella migrazione dal codice proposto al codice con socket iterativo.

N.B. Le funzioni della libreria socket devono essere proposte in modo completo con **tutti** i parametri specificati. Non verrà accettato uno pseudocodice che utilizza le librerie socket di Java.

Esercizio 2) (12 punti)

(Per studenti in presenza dall'A.A. 2009/10 in poi) Si consideri l'applicazione prenotazione (Esercizio 1) e si supponga di volerla implementare tramite RPC. Si proponga e descriva riga per riga l'interfaccia IDL del server RPC. Si discutano inoltre nel dettaglio come vengono gestiti i malfunzionamenti RPC e le diverse semantiche di chiamata.

(Per studenti online e per studenti in presenza A.A. precedente al 2009/10) Nell'ambito del protocollo SMTP, si proponga un esempio di messaggio di posta con estensione MIME multipart che contiene un testo e un'immagine. Si descriva inoltre il flusso di richieste e risposte POP3 per *i)* la ricerca di tale messaggio nella casella di posta `nomestudente@studenti.unimi.it`, *ii)* la lettura di tale messaggio e *iii)* la sua cancellazione dopo la lettura.

Domanda 1) (3 punti)

Nell'ambito del protocollo FTP si discutano le differenze tra FTP in modalità attiva e modalità passiva.